G大脑袋 × Tangsir 工作对话纪要

日期: 2026-03-28（周六） **对话范围**: Doramagic 项目研究 + Skill 锻造 + 架构设计讨论 **模型**: Claude Opus 4.6 (thinking mode) → 实际执行用 MiniMax

---

一、Session 启动

G大脑袋: 周六上午好，报告当前使用 Opus 思考模式，日常任务自动切 MiniMax 省 token。问 Tangsir 有什么想聊的。

二、Doramagic v12 研究 + DeerFlow 深度调研

2.1 Tangsir 发起研究任务

Tangsir: 发了两个 GitHub 链接： – `https://github.com/bytedance/deer-flow/tree/main` — 字节跳动的开源项目 – `https://github.com/tangweigang-jpg/Doramagic` — Tangsir 自己开发的开源项目

Tangsir 描述的两个核心痛点：

> 1. **黑盒问题**：给 Doramagic 命令，低能力大模型干活儿去了，执行过程中不在 OpenClaw 进行过程汇报，体感很沮丧，不知道发生了什么，无法预期结果。
> 2. **线性流程问题**：现在设计的 Doramagic 是线性流程，增加失败风险，也不利于提高效率。

Tangsir 目标： – 深度研究 DeerFlow，找出好的解决方案 – 对 Doramagic 提出 10+ 技术工程建议 – 另外请审阅 "web-access"（复杂的 AI 开源项目） – 输出硅谷技术专家级建议报告

2.2 G大脑袋执行深度研究

研究范围： – DeerFlow 2.0 完整架构（CLAUDE.md、README 中英文、SubagentExecutor 源码、中间件链） – Doramagic 源码（phase_runner.py、SKILL.md、README） – browser-use 项目（Web 访问/浏览器自动化框架）

DeerFlow 2.0 核心发现： 1. **Super Agent Harness 定位**：不只是框架，是运行时基础设施 2. **12 层中间件链**：可组合、可测试、每层单一职责 3. **SubagentExecutor 流式执行**：用 `astream()` + `SubagentResult` 实时捕获每个 AI 消息 4. **DAG 执行模型**：基于 LangGraph，节点可并行，条件边动态路由 5. **双线程池架构**：

`_scheduler_pool`（3 workers）调度，`_execution_pool`（3 workers）执行 6. **trace_id 追踪**：父代理 trace_id 传递给子代理，全链路可观测 7. **Harness/App 分层**：框架代码和应用代码严格隔离，CI 强制检查

2.3 产出：DeerFlow 深度研究 × Doramagic 技术建议报告（PDF 已发送）

15 条技术建议（按优先级）：

P0（立即实施）： 1. Event Bus 实时进度汇报系统 → 解决黑盒问题 2. DAG 并行管线替代线性管线 → 解决线性流程问题 3. 多项目并行提取 → 效率提升 40-60%

P1（短期实施）： 4. Checkpoint/Resume 断点续跑机制 5. 中间件链架构重构 6. 分布式追踪（trace_id 传播） 7. Harness/App 分层重构 8. 智能错误恢复

P2（中期实施）： 9. browser-use 集成增强社区信号采集 10. MCP Server 接口 11. 流式输出组装 12. 上下文工程优化 13. 长期记忆系统 14. 质量门控增强（Guardrail） 15. Embedded Python Client

---

三、Doramagic v12.1.1 管线架构重构设计

3.1 Tangsir 提供额外上下文

Tangsir 提供了 8 条真实问题： 1. 用户给了 GitHub URL，系统仍跑 Discovery（浪费） 2. 用户请求模糊，不追问直接盲搜（质量差） 3. awesome-list 和深度框架走同一条提取路径（错误） 4. 3 个项目串行提取，一个失败污染全局 5. 编译占总耗时 51%（138s/268s） 6. 编译完才发现质量差，前 4 分钟白费 7. 任何失败 = 整个 run 失败，用户只看到"失败" 8. 执行 4 分钟黑盒无反馈

Tangsir 提供的 Doramagic 已有资产： - **v7.0 三层架构**：Controller（12 状态 FSM）→ Executor（8 个独立执行器）→ Platform Adapter - **Lease Token 门控**：每个 phase 获取 lease 才能执行，TTL 过期自动降级 - **8 条血泪教训**： 1. skill name 就是路由 key，改名 = 全部调用断裂 2. OpenClaw exec 工具不可用，必须用 subprocess 3. LLM 会逃逸预设角色，必须结构化约束 4. 控制权不能交给 LLM，必须代码控制流程骨架 5. 信息饥饿：上游阶段不传足够信息给下游 = 下游输出质量暴跌 6. 三方研究法（Claude+Codex+Grok）是发现盲点的最有效方式 7. 砖块注入必须有域隔离，否则无关领域知识污染提取 8. 必须有端到端冒烟测试

- **外部框架启发**：
- LangGraph：图状态机、TypedDict+Reducer、Send() API 动态扇出、Checkpointer
- DSPy：编译范式、Module 声明式、Optimizer 自动调优 prompt
- CrewAI：任务输出截断是常见 silent failure、无限循环防护
- LlamaIndex：RouterQueryEngine 按输入类型路由、数据流隔离
- **Haystack 2.0**：Pipeline.connect() 显式声明数据流、ConditionalRouter 用 Jinja2 做条件路由、支持 Fan-out/Fan-in/Loop

3.2 G大脑袋输出 v12.1.1 架构设计方案

核心改造：确定性上下文路由（Deterministic Context Routing），基于 `PipelineContext` 动态路由，替代固定线性 TRANSITIONS。

7 大设计模块：

1. **输入路由器（Input Router）**：4 象限分类（URL 直达/明确项目名/泛领域探索/意图模糊），确定性路由标志，模糊意图触发 CLARIFY 状态暂停管线请求用户补充
2. **状态机改造与条件边**：Stage 0 后加入判定（Markdown 占比 > 80% → `is_collection=True`），awesome-list 类项目路由到轻量级元数据提取路径
3. **扇出/扇入与数据隔离**：Fan-out：N 个独立 `CD_Executor` 子进程，独立 staging 目录。Fan-in：只要 ≥1 个成功就继续，失败记录到 `context.failed_repos`
4. **信息传递契约**：严格定义上下游 Payload Schema（A→B 传 `SearchConstraints`，B→CD 传 `RepoContext`，CD→E 传结构化卡片+DSD 分数）
5. **质量反馈环与模块化编译**：Pre-flight Check（E 结束时规则引擎快检，不达标直接降级不进入高耗时 F）。针对性修复：G 报错指出 `weakest_module`，仅将该模块发回 F_PARTIAL 重新编译
6. **降级交付体系**：Level 1（最佳）/ Level 2（编译失败降级到原始卡片库）/ Level 3（提取部分失败降级到独立报告）/ Level 4（发现失败降级到项目清单）
7. **进度事件架构**：`run_events.jsonl` Schema，Platform Adapter 每 3-5 秒向 Telegram 推送状态更新

四、Business Skill Finder Skill 锻造

4.1 Tangsir 提出需求

Tangsir: "我需要一个让我方便寻找 skill 的工具" + "商业人士，办公室人士"

4.2 G大脑袋分析与执行

分析：Doramagic 核心价值是从代码库提取暗知识。ClawHub 目前几乎没有公开 skill，直接跑完整提取流程价值有限。

决策：基于对 OpenClaw skill 生态的深度了解，直接设计 Business Skill Finder Skill。

4.3 产出的 Skill

安装位置: `~/.openclaw/skills/business-skill-finder/`

核心能力： – 🎯 智能匹配：基于工作场景推荐最合适的 OpenClaw 技能 – 📋 场景覆盖：文档处理 / 邮件协作 / 竞品研究 / 自动化 / 写作 / 健康管理 – ⚠️ 使用顾问：告知禁忌、入手成本、如何开始 – 🗣️ 双语触发：支持中文和英文触发词

使用方式：

```
/business-skill-finder 我是产品经理，想提高团队协作效率
```

测试场景：用"季度销售汇报"演示，推荐 gws-sheets + pdf-to-ppt 组合，并说明需要 Google Workspace 认证才能真实运行。

五、报告归档位置

所有报告保存在: `~/Documents/openclaw/Doramagic/Research/`

文件名	内容
`20260328_DeerFlow深度研究与Doramagic技术建议.md`	DeerFlow 调研 + 15 条技术建议
`20260328_Doramagic_v12_1_1_架构重构设计报告.md`	v12.1.1 管线重构设计方案

六、后续可执行事项

- [ ] 配置 gws-sheets 的 Google Workspace 认证（需要真实 OAuth 凭据）
- [ ] v12.1.1 架构设计方案是否启动开发?
- [ ] Doramagic 跑一次真实 Skill 锻造流程测试

纪要整理于 2026-03-28 22:00 GMT+8